Evaluating Distributed Processing on Cloud: A Comparative Study of Hadoop and Spark

Andrea Innocenti, Savino Gorgoglione, Domenico Lenzerini

19 June 2025

Abstract

This paper presents a comparative evaluation of Hadoop MapReduce, Apache Spark, and non-parallel Python implementations for the construction of an inverted index, a fundamental problem in information retrieval. Performance is analyzed with respect to input size, number of reducers, and available compute resources, using metrics such as execution time, shuffle overhead, CPU utilization, and memory consumption. The findings are presented through comprehensive plots and tables.

1 Dataset

The data was collected using two custom scraping scripts [10, 11] applied to texts from *Gutenberg* [1], a public online library of over 75,000 ebooks in plain text format. In particular the texts are from the sections *Biographies*, *Architecture*, *Archaeology*, *Medicine*, *Business*, *Children and Adult reading*, *Music*, *Nutrition*, *Sociology*, *Travel* and *Literature*. The original dataset, comprising text files of various sizes (from a few KB to several MB) totaling 2.1 GB, was merged into a single large file. Subsequently, this file was split into smaller segments of variable dimension, ranging between 500kb, 1mb, 5mb, 25mb, 50mb for a total of 61 text files. This can be replicated using [9]. This segmentation makes it easier to evaluate how the programs perform when processing inputs of varying dimensionality.

2 Pseudocode

Before evaluating the performance of the various approaches used to solve the inverted index problem, the pseudocode for the Hadoop Map and Reduce solution is presented. Listing 1 presents the pseudocode of the Mapper class for the Hadoop MapReduce framework designed to build an inverted index. The InvertedIndexMapperOpt class processes each input line by first normalizing it to lowercase and removing any non-alphanumeric characters except spaces. It then tokenizes the cleaned line into individual words. Common stopwords (predefined filler words that do not contribute meaningfully to the index) are excluded from processing. For each valid word, the Mapper emits a key combining the word itself and the source filename, paired with a value of one.

```
1    CLASS InvertedIndexMapperOpt EXTENDS Mapper<LongWritable, Text, Text,
         IntWritable>:
2
3    PRIVATE filename : String
4    PRIVATE final one : IntWritable = new IntWritable(1)
5    PRIVATE final wordAtFile : Text = new Text()
6
7    PRIVATE static final STOPWORDS : Set<String> = new HashSet<>(Arrays.asList(
         "the", "is", ... , "and"))
```

```
8
9       METHOD setup(context : Context):
10          split : FileSplit = (FileSplit) context.getInputSplit()
11          path : Path = split.getPath()
12          filename = path.getName()
13      END METHOD
14
15      METHOD map(key : LongWritable, value : Text, context : Context):
16          line : String = value.toString().toLowerCase().replaceAll("[^a-z0-9 ]",
                " ")
17          tokenizer : StringTokenizer = new StringTokenizer(line)
18
19          WHILE tokenizer.hasMoreTokens():
20              word : String = tokenizer.nextToken()
21              IF NOT word.isEmpty() AND NOT STOPWORDS.contains(word):
22                  wordAtFile.set(word + "@" + filename)
23                  context.write(wordAtFile, one)
24              END IF
25          END WHILE
26      END METHOD
27
28  END CLASS
```

Listing 1: Mapper code

A Combiner logic was implemented, as shown in Listing 2. The InvertedIndexCombiner class defines a simple Combiner for the Hadoop MapReduce framework. In this implementation, the reduce method receives a key of type Text (consisting of a word and its source file name, as mentioned before) and an iterable of IntWritable values, each representing a count of 1 performed by the Mapper. The method computes the sum of these values using a local accumulator and emits the resulting key-value pair. This early aggregation reduces the volume of intermediate data transmitted during the shuffle and sort phase, thereby lowering network overhead and improving overall performance, especially when the same word appears multiple times within the same input split.

```
1       CLASS InvertedIndexCombiner EXTENDS Reducer<Text, IntWritable, Text,
            IntWritable>:
2
3       METHOD reduce(key : Text, values : Iterable<IntWritable>, context : Context
            ):
4           sum : Integer = 0
5           FOR each val : IntWritable IN values:
6               sum = sum + val.get()
7           END FOR
8           context.write(key, new IntWritable(sum))
9       END METHOD
10
11  END CLASS
```

Listing 2: Combiner code

The InvertedIndexReducer class, shown in Listing 3, completes the inverted index construction by aggregating and formatting the intermediate data received from the Mapper and already reduced by the Combiner. For each key, representing a unique combination of a word and the corresponding filename, the reduce method splits the key to separate the word from the filename, then sums all associated integer values to compute the total number of occurrences of that word in the given file. This information is stored in a data structure, invertedIndex, which maps each word to a list of strings in the format filename:count. Once all input data has been processed, the cleanup method is invoked to emit the final output. For each word in the invertedIndex map, it concatenates all associated file-count entries into a single string and writes the word along

with this list to the context. This ensures that the final output correctly maps each word to the list of documents in which it appears, along with its frequency, thereby producing a complete inverted index.

```
CLASS InvertedIndexReducer EXTENDS Reducer<Text, IntWritable, Text, Text>:

    PRIVATE invertedIndex : Map<String, List<String>> = new HashMap<>()

    METHOD reduce(key : Text, values : Iterable<IntWritable>, context : Context
        ):
        parts : ARRAY OF String = key.toString().split("@")
        word : String = parts[0]
        filename : String = parts[1]

        sum : Integer = 0
        FOR each val : IntWritable IN values:
            sum = sum + val.get()
        END FOR

        entry : String = filename + ":" + sum

        IF NOT invertedIndex.containsKey(word):
            invertedIndex.put(word, new ArrayList<>())
        END IF
        invertedIndex.get(word).add(entry)

    END METHOD

    METHOD cleanup(context : Context):
        FOR each entry : Map.Entry<String, List<String>> IN invertedIndex.
            entrySet():
            word : String = entry.getKey()
            files : String = String.join(" ", entry.getValue())
            context.write(new Text(word), new Text(files))
        END FOR
    END METHOD

END CLASS
```

Listing 3: Reducer code

Besides the standard MapReduce approach described earlier, an alternative design pattern known as in-mapper combining is also presented. The InvertedIndexMapper_inMapperCombiner class, shown in Listing 4, aims to reduce the volume of intermediate key-value pairs emitted by the Mapper. Unlike the traditional setup, in which the Mapper emits a record for every valid word occurrence and relies on a separate Combiner class to perform partial aggregation, this implementation accumulates word counts directly within the Mapper using a local HashMap (inMapperCounts). During the map phase, instead of immediately emitting intermediate output, the Mapper updates the local count for each encountered word–filename pair. In the cleanup method, executed once at the end of the Mapper task, it emits a single key-value pair per distinct word–filename combination. This strategy significantly reduces the number of intermediate outputs and the associated communication overhead during the shuffle and sort phase, thereby improving efficiency in scenarios with high repetition of terms within individual input splits.

```
    CLASS InvertedIndexMapper_inMapperCombiner EXTENDS Mapper<LongWritable,
        Text, Text, IntWritable>:

    PRIVATE filename : String
    PRIVATE final inMapperCounts : Map<String, Integer> = new HashMap<>()
    PRIVATE final wordAtFile : Text = new Text()

```

```
    PRIVATE static final STOPWORDS : Set<String> = new HashSet<>(Arrays.asList(
        "the", "is", ..., "and"))

    METHOD setup(context : Context):
        split : FileSplit = (FileSplit) context.getInputSplit()
        path : Path = split.getPath()
        filename = path.getName()
    END METHOD

    METHOD map(key : LongWritable, value : Text, context : Context):
        line : String = value.toString().toLowerCase().replaceAll("[^a-z0-9 ]",
            " ")
        tokenizer : StringTokenizer = new StringTokenizer(line)

        WHILE tokenizer.hasMoreTokens():
            word : String = tokenizer.nextToken()
            IF NOT word.isEmpty() AND NOT STOPWORDS.contains(word):
                mapKey : String = word + "@" + filename
                inMapperCounts.merge(mapKey, 1, Integer::sum)
            END IF
        END WHILE
    END METHOD

    METHOD cleanup(context : Context):
        FOR each entry : Map.Entry<String, Integer> IN inMapperCounts.entrySet
            ():
            wordAtFile.set(entry.getKey())
            context.write(wordAtFile, new IntWritable(entry.getValue()))
        END FOR
    END METHOD

END CLASS
```

Listing 4: InMapperCombining code

3 Performance evaluation

The *Inverted Index* problem is addressed using four different implementation strategies: the standard `Hadoop MapReduce` approach with a `Combiner` [2][3][5][6](pseudocode shown in Listings 1, 2, and 3), a `Hadoop MapReduce` implementation with `In-Mapper Combining` [4] (pseudocode shwon in Listing 4), an `Apache Spark`-based implementation [7], and a `Non-Parallel Python` implementation [8]. The behavior of the aforementioned methods is evaluated over input sizes of 5 MB, 50 MB, 200 MB, 1 GB, 2 GB. All performance statistics for each program execution are reported as the average of three independent runs, all conducted on a cluster composed of three machines, each equipped with 2 CPU cores and 6.8 GB of RAM. To facilitate the exploration of the output produced by each implementation, formatted as a list of entries in the form `[Word Document_Name_1:Count Document_Name_2:Count ...]`, a Python script was developed to perform keyword-based search queries [12]. The script allows users to input either a single word or a sequence of words (even a sentence). In the first case, the script returns the names of the documents in which the word appears. In the second case, it returns only those documents that contain all the queried words. It's possible to perform multiple queries in just one execution of the program. Additionally, the search process ignores stopwords, in alignment with the filtering applied in the respective indexing implementations. All the analysis included in the following subsections are performed using scripts presented in this Jupyter Notebook [13].

3.1 Standard Hadoop MapReduce: Execution Statistics with Varying Number of Reducers

The `Total Execution Time`, `Average Map Phase Time`, `Average Shuffle Phase Time`, and `Average Reduce Phase Time` are analyzed with respect to varying input sizes and different numbers of reducers. Initially, the tests are executed with input files split into very small chunks (1 MB each), as shown in the aforementioned Jupyter Notebook, where it is evident that the computational times increase significantly. This is primarily due to the growth in the number of input splits, which leads to a proportional increase in the number of mapper tasks. Therefore, the following analysis are conducted using the split presented in Section 1, that reflects a more realistic use case for an Hadoop environment.

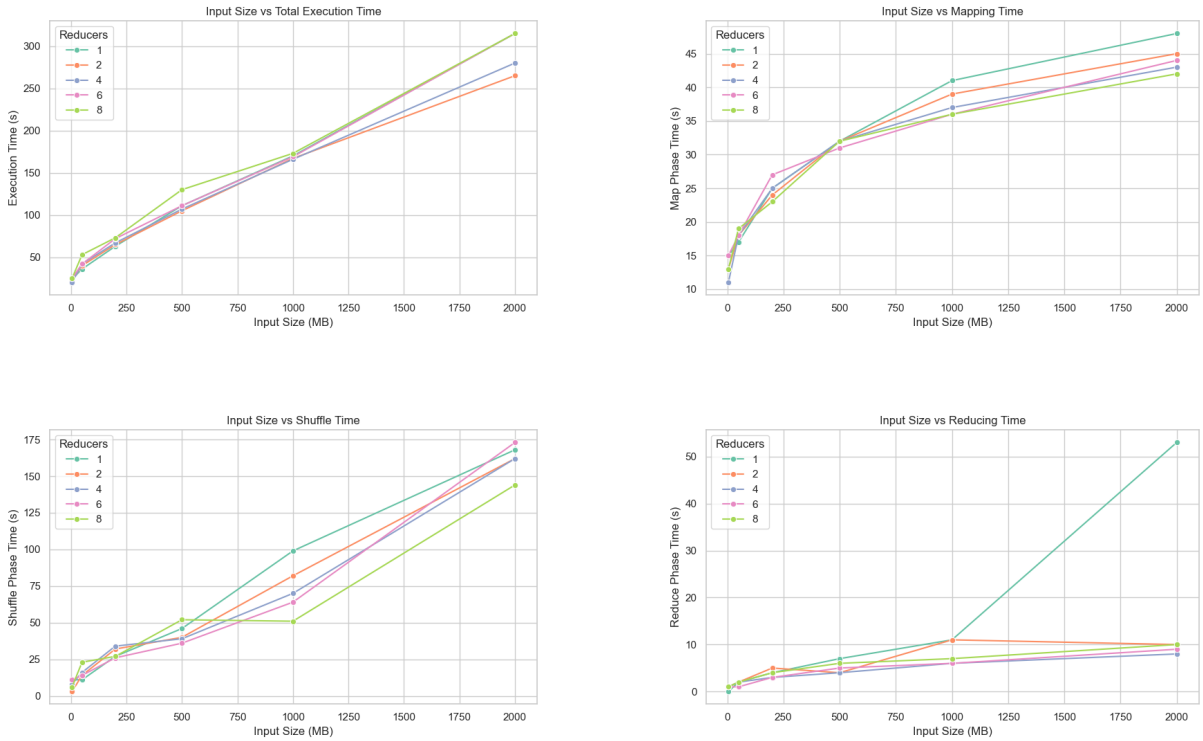


Figure 1: Performance Analysis with varying input sizes and different number of reducers.

Considering the selected input sizes and the hardware configuration, increasing the number of reducers beyond two does not appear to yield further performance improvements. The most notable difference across the various plots is observed in the reduce phase, where the average reduce time significantly decreases when scaling from one to two reducers.

3.2 Standard Hadoop MapReduce vs In-Mapper Combining Hadoop MapReduce: Execution Statistics

Following, `Total Execution Time` and `Average Shuffle Phase Time` are analyzed with respect to varying input sizes and different Mapper-Combiner design strategies. The performance evaluation primarily focuses on `Total Execution Time`, while the `Shuffle Phase Time` plot is included due to its expected significant variation between the different approaches. The In-Mapper Combining MapReduce implementation is conducted using a single reducer and compared against both the best and worst configurations previously evaluated for the Standard MapReduce.

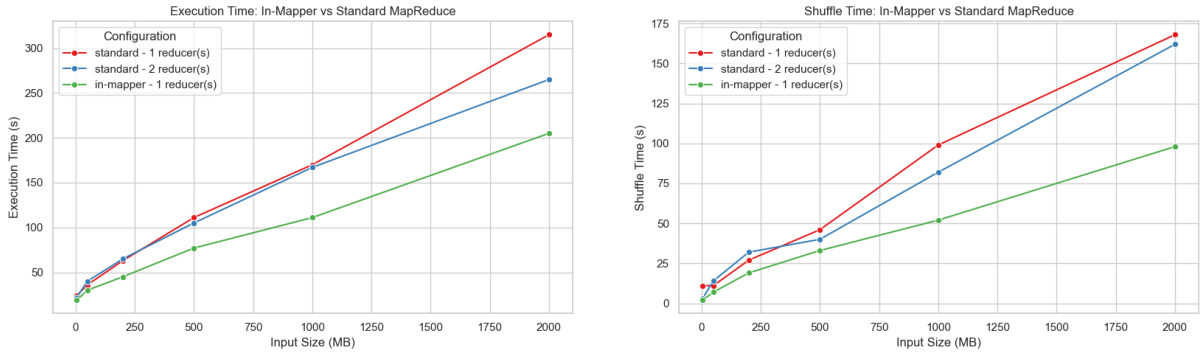
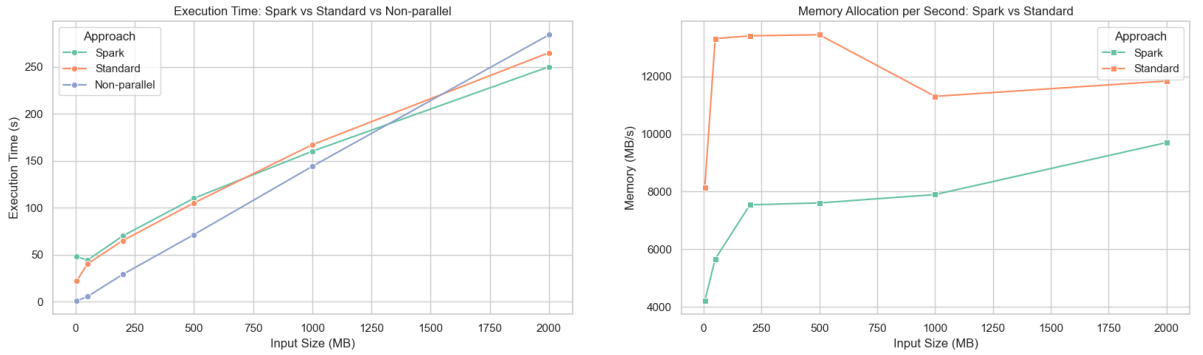


Figure 2: Performance Analysis with varying input sizes and design strategies.

As expected, the In-Mapper Combining solution delivers a significant improvement in Total Execution Time, primarily due to the reduction in Shuffle Phase Time.

3.3 Standard Hadoop MapReduce vs Non-Parallel Python vs Apache Spark: Execution Statistics

Finally, `Total Execution Time` is analyzed with respect to varying input sizes and different computational approaches to the Inverted Index problem. In the Spark implementation [7], documents are loaded as (file_path, content) pairs using `wholeTextFiles`, preserving filenames for indexing. After cleaning and tokenization, words (excluding stopwords) are paired with their filenames as `word@filename`. A local combiner via `mapPartitions` aggregates counts within each partition, significantly reducing intermediate data. The final global count is computed with `reduceByKey`, and results are grouped to build the inverted index. For both Spark and Hadoop MapReduce, additional statistics on `Average vCore Usage` and `Average Memory Allocation` are examined. It should be noted, however, that differences in configuration between the two frameworks, or possibly differences in how resource metrics are calculated within HDFS, may contribute to the significant variation in resource utilization observed in the subsequent plots.



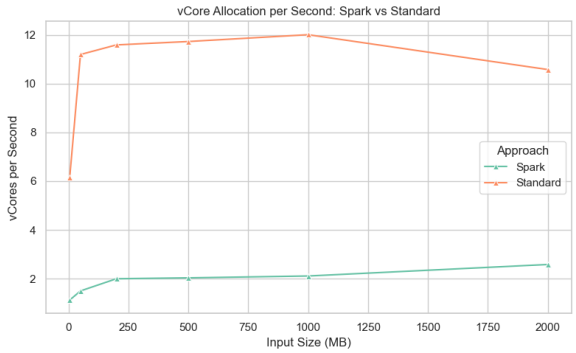


Figure 3: Performance Analysis with varying input size and different computational approaches.

4 References

References

[1] Project Gutenberg, https://www.gutenberg.org.

[2] Driver code, https://github.com/andreainno22/Inverted_index_project/blob/main/src/main/java/org/Inverted_index/InvertedIndexDriver.java

[3] Map code, https://github.com/andreainno22/Inverted_index_project/blob/main/src/main/java/org/Inverted_index/InvertedIndexMapperOpt.java

[4] InMapperCombining code, https://github.com/andreainno22/Inverted_index_project/blob/main/src/main/java/org/Inverted_index/InvertedIndexMapper_inMapperCombiner.java

[5] Combiner code, https://github.com/andreainno22/Inverted_index_project/blob/main/src/main/java/org/Inverted_index/InvertedIndexCombiner.java

[6] Reducer code, https://github.com/andreainno22/Inverted_index_project/blob/main/src/main/java/org/Inverted_index/InvertedIndexReducer.java

[7] Spark code, https://github.com/andreainno22/Inverted_index_project/blob/main/invInd_spark_opt.py

[8] Non-parallel Python code, https://github.com/andreainno22/Inverted_index_project/blob/main/Inverted_index.py

[9] Python Code to split block of text data, https://github.com/andreainno22/Inverted_index_project/blob/main/split_file.py.

[10] Python Code to perform data scraping in frequent downloaded texts page, https://github.com/andreainno22/Inverted_index_project/blob/main/webScraping/download_freq_downloaded.py.

[11] Python Code to perform data scraping in Site Directories, https://github.com/andreainno22/Inverted_index_project/blob/main/webScraping/download_from_dir.py.

[12] Search query code, https://github.com/andreainno22/Inverted_index_project/blob/main/Query_for_all_scripts.py

[13] Jupyter Notebook for testing, https://github.com/andreainno22/Inverted_index_project/blob/main/CloudPerfAnalysis.ipynb